B.TECH · 1ST YEAR · 1ST SEMESTER

DATA STRUCTURES

Unit III — Stacks

Formula Sheet & Revision Document

Complete exam-oriented notes · Algorithms · Pseudocode · Complexity

Last-day revision weapon · 15 sections

TABLE OF CONTENTS

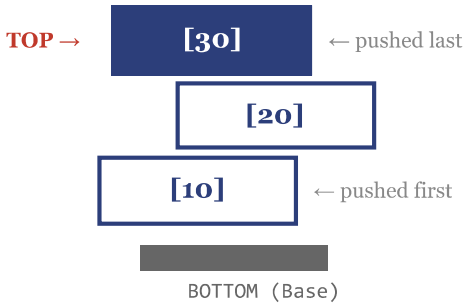
01 Introduction to Stacks

Definition

A **Stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle — the element inserted last is the first to be removed.

It is a restricted data structure with access only at one end called the **TOP**.

Stack Diagram



LIFO Principle

LAST IN, FIRST OUT (LIFO)

The most recently added element is always the first to be removed — like a stack of plates.

- Push → Inserts at TOP
- Pop → Removes from TOP
- New elements sit ON TOP of old ones

Key Properties

- ✅ Single access point — the **TOP**
- ✅ Restricted access — no random indexing
- ✅ Order reversal behavior (LIFO)
- ✅ Overflow when full, Underflow when empty
- ✅ Can be implemented with arrays or linked lists

Real-World Analogies

- 🍽️ **Stack of plates** — last plate placed is first removed
- ↩️ **Undo/Redo** in editors — last action undone first
- 📚 **Books stacked** — bottom book is hardest to reach
- 🌐 **Browser back button** — last visited page first

02 Representation of Stack

Stack Using Arrays

ARRAY REPRESENTATION

- Uses contiguous memory blocks
- Index-based access
- Fixed size (declared at compile time)
- Variables: `stack[]`, `top = -1`, `MAX`

```
Index:  0    1    2    3
       [10] [20] [30] [ ]
                 ↑
              top = 2
MAX = 4, Size = 4
```

Stack Using Linked List

LINKED LIST REPRESENTATION

- Dynamic size — grows/shrinks at runtime
- Pointer-based (nodes linked via `next`)
- No fixed capacity
- `top` pointer → first node (most recent)

```
top
 ↓
[30|→] → [20|→] → [10|NULL]
 (head)
 newest                oldest
```

STACK VARIABLES

Variable	Meaning
`top`	Index of top element (initially −1)
`MAX / size`	Maximum capacity of the stack
`stack[]`	Array holding stack elements

COMPARISON SUMMARY

Property	Array	Linked List
Size	**Fixed**	**Dynamic**
Memory	Contiguous	Scattered
Overhead	**None**	**Pointers**
Speed	**Fast**	Moderate

03 Stack Operations — Complete Reference

🔼 PUSH — Insert Element

ALGORITHM (STEPS)

1. Check if `top == MAX - 1`
2. If YES → Print **Overflow**, stop
3. Increment `top` by 1
4. Set `stack[top] = x`

PSEUDOCODE

```
PUSH(stack, x):
  if top == MAX-1:
      print "Overflow"
  else:
      top ← top + 1
      stack[top] ← x
```

EXAMPLE

Push 40 → top becomes 3, stack[3] = 40

`O(1)`

🔽 POP — Remove Element

ALGORITHM (STEPS)

1. Check if `top == -1`
2. If YES → Print **Underflow**, stop
3. Save `val = stack[top]`
4. Decrement `top` by 1
5. Return `val`

PSEUDOCODE

```
POP(stack):
  if top == -1:
      print "Underflow"
  else:
      val ← stack[top]
      top ← top - 1
      return val
```

EXAMPLE

Pop from [10,20,30] → returns 30, top = 1

`O(1)`

👁 PEEK / TOP — View Top Element

ALGORITHM

1. Check if `top == -1` (empty)
2. If YES → Print "Stack Empty"
3. Return `stack[top]` (do NOT remove)

```
PEEK(stack):
  if top == -1:
      print "Empty"
  else:
      return stack[top]
```

`O(1)`

✅ isEmpty & isFull

ISEMPTY

```
isEmpty(stack):
  if top == -1:
      return TRUE
  return FALSE
```

ISFULL

```
isFull(stack):
  if top == MAX-1:
      return TRUE
  return FALSE
```

`O(1)` both operations

COMPLEXITY TABLE — ALL OPERATIONS

Operation	Time Complexity	Space Complexity	Condition
Push	O(1)	O(1)	Stack not full
Pop	O(1)	O(1)	Stack not empty
Peek / Top	O(1)	O(1)	Stack not empty
isEmpty	O(1)	O(1)	Always
isFull	O(1)	O(1)	Array only
Search	O(n)	O(1)	Linear scan
Traverse	O(n)	O(1)	Visit all elements
Space (overall)	—	O(n)	n = number of elements

04 Implementation Using Arrays (Static)

Complete Array Stack in C

```c
#define MAX 100

int stack[MAX];
int top = -1;

/* PUSH */
void push(int x) {
    if (top == MAX-1)
        printf("Overflow!\n");
    else
        stack[++top] = x;
}

/* POP */
int pop() {
    if (top == -1) {
        printf("Underflow!\n");
        return -1;
    }
    return stack[top--];
}

/* PEEK */
int peek() {
    if (top == -1) return -1;
    return stack[top];
}
```

Overflow & Underflow Conditions

OVERFLOW

Occurs when pushing into a **full** stack.

Condition: `top == MAX - 1`

UNDERFLOW

Occurs when popping from an **empty** stack.

Condition: `top == -1`

Advantages vs Disadvantages

✅ Advantages	❌ Disadvantages
Simple to implement	Fixed size (static)
Fast O(1) access	Memory wastage
No pointer overhead	Overflow possible
Cache-friendly	Resize not easy

05 Implementation Using Linked List (Dynamic)

Node Structure

```c
struct Node {
    int data;
    struct Node* next;
};

struct Node* top = NULL;
```

Linked List Stack Diagram

Push (Linked List)

```c
void push(int x) {
    Node* newNode =
        (Node*)malloc(sizeof(Node));
    newNode→data = x;
    newNode→next = top;
    top = newNode;
}
```

Pop (Linked List)

```c
int pop() {
    if (top == NULL)
        printf("Underflow!\n");
    int val = top→data;
    Node* temp = top;
    top = top→next;
    free(temp);
    return val;
}
```

```
After pushing 10, 20, 30:

top
 ↓
[30 | •]──→[20 | •]──→[10 | NULL]

Push 40:
top
 ↓
[40 | •]──→[30 | •]──→[20 | •]──→[10 | NULL]

Pop:
Removes 40, top moves to [30|•]
```

Advantages vs Disadvantages

✅ Advantages	❌ Disadvantages
Dynamic size	Pointer overhead
No overflow (heap)	More memory per node
Efficient memory use	Slower allocation
Easy grow/shrink	Not cache-friendly

06 Array vs Linked List Stack — Full Comparison

Feature	Array Stack	Linked List Stack	Winner
Size	**Fixed at compile time**	**Dynamic, grows at runtime**	Linked List
Memory Layout	Contiguous blocks	Non-contiguous (scattered)	Array (cache)
Overflow Risk	**Yes (when top = MAX−1)**	**Rare (only if heap full)**	Linked List
Underflow Risk	Yes (top = −1)	Yes (top = NULL)	Tie
Push Complexity	**O(1)**	**O(1)**	Tie
Pop Complexity	**O(1)**	**O(1)**	Tie
Memory per element	**Just the data**	**Data + pointer**	Array
Implementation	**Simple**	Moderate	Array
Resizing	**Not possible (static)**	**Automatic**	Linked List
Best Use Case	Known size, speed needed	Unknown/variable size	Depends

07 Applications of Stacks

EXPRESSION EVALUATION

- Infix → Postfix conversion
- Postfix expression evaluation
- Infix → Prefix conversion
- Operator precedence handling

FUNCTION CALLS

- Recursion uses call stack
- Local variables stored
- Return addresses saved
- Stack frame per function call

UNDO / REDO

- Text editors (Ctrl+Z)
- Two stacks: undo + redo
- Push action to undo stack
- Pop to undo, push to redo

BACKTRACKING

- Maze solving algorithms
- Depth First Search (DFS)
- N-Queens problem
- Sudoku solvers

BALANCED PARENTHESES

- Validate { } [ ] ( )
- Push opening brackets
- Pop & match closing
- Used in compilers

REVERSALS

- Reverse a string
- Reverse a linked list
- Reverse a number's digits
- Convert decimal to binary

Balanced Parentheses — Algorithm

DFS using Stack

```
isBalanced(expr):
  stack ← empty
  for each char c in expr:
    if c ∈ { '(' '[' '{' }:
        push(c)
    elif c ∈ { ')' ']' '}' }:
        if isEmpty():
            return FALSE
        top ← pop()
        if !matches(top, c):
            return FALSE
  return isEmpty()
```

```
DFS(graph, start):
  stack ← empty
  visited ← {}
  push(start)
  while !isEmpty():
    node ← pop()
    if node ∉ visited:
        visit(node)
        visited.add(node)
        for neighbor in graph[node]:
            push(neighbor)
```

08 Expression Evaluation (Important)

Expression Types

Type	Notation	Example
Infix	Operator between operands	`A + B`
Prefix	Operator before operands	`+ A B`
Postfix	Operator after operands	`A B +`

Operator Precedence & Associativity

Operator	Precedence	Associativity
`^ (power)`	Highest (3)	Right
`* /`	Medium (2)	Left
`+ -`	Low (1)	Left
`( )`	Lowest (0)	—

Common Conversions

Infix	Postfix	Prefix
`A + B`	`AB+`	`+AB`
`A + B * C`	`ABC*+`	`+A*BC`
`(A + B) * C`	`AB+C*`	`*+ABC`
`A * B + C / D`	`AB*CD/+`	`+*AB/CD`

Infix → Postfix Algorithm

1. Scan expression left to right
2. If operand → append to output
3. If `(` → push to stack
4. If `)` → pop until `(`, append
5. If operator → pop while stack-top has ≥ precedence, then push
6. After scan → pop all remaining operators

Example: A + B * C – D

Symbol	Stack	Output
A		A
+	+	A
B	+	AB
*****	+*	AB
C	+*	ABC
–	–	ABC*+
D	–	ABC*+D
end		ABC*+D–

Result: `ABC*+D–`

Postfix Evaluation Algorithm

```
evaluatePostfix(expr):
  stack ← empty
  for each token t in expr:
    if t is operand:
        push(t)
    else (t is operator):
        b ← pop()   // right operand
        a ← pop()   // left operand
        result ← a t b
        push(result)
  return pop()
```

Infix → Prefix Algorithm

1. Reverse the infix expression
2. Swap `(` with `)` and vice versa
3. Apply Infix → Postfix algorithm
4. Reverse the output → that's the Prefix

🧠 QUICK MEMORY TRICK

- **Infix**: Human-readable (A + B)

Example: 5 3 2 * +

Token	Action	Stack
5	push	5
3	push	5, 3
2	push	5, 3, 2
*	pop 2,3 → 3*2=6 → push 6	5, 6
+	pop 6,5 → 5+6=11 → push 11	11

Result = 11

- **Postfix**: Computer-friendly, no brackets needed
- **Prefix**: Used in LISP, functional languages
- Postfix evaluation uses stack — pop 2, operate, push result
- Infix→Postfix uses stack for operators only

Function Call Stack

Each function call creates a **stack frame** containing:

- Local variables
- Parameters
- Return address

```
main() → foo() → bar()

Stack:
[bar() frame  ] ← TOP
[foo() frame  ]
[main() frame ] ← BOTTOM
```

09 Time & Space Complexity Summary

Operation	Array Stack	LL Stack
Push	O(1)	O(1)
Pop	O(1)	O(1)
Peek/Top	O(1)	O(1)
Search	O(n)	O(n)
Traverse	O(n)	O(n)
Space	O(n)	O(n)

Comparison with Other Structures

Structure	Insert	Delete	Search	Access
Stack	O(1)	O(1)	O(n)	O(n)
Queue	O(1)	O(1)	O(n)	O(n)
Array	O(n)	O(n)	O(n)	O(1)
BST	O(log n)	O(log n)	O(log n)	O(n)

GOLDEN RULE

All primary stack operations (Push, Pop, Peek, isEmpty, isFull) are **O(1) — constant time**. This is because we always operate at the TOP — no traversal needed. Only Search and Traverse require O(n).

10 Key Terms & Definitions (30+ Terms)

Stack	Linear LIFO data structure	**LIFO**	Last In, First Out principle
TOP	Index/pointer to topmost element	**Push**	Insert element at the top
Pop	Remove element from the top	**Peek**	View top element without removing
Overflow	Push on a full stack	**Underflow**	Pop from an empty stack
isEmpty	Checks if stack has no elements	**isFull**	Checks if stack is at capacity
MAX	Maximum capacity of array stack	**Infix**	Operator between two operands: A+B
Postfix	Operator after operands: AB+	**Prefix**	Operator before operands: +AB
Stack Frame	Memory block for one function call	**Call Stack**	Stack of active function frames
Recursion	Function calling itself, uses stack	**DFS**	Depth First Search — uses stack
Backtracking	Undo choices using stack	**Precedence**	Priority of operators: ^ > * / > + –
Associativity	Left/right evaluation order	**Node**	Single element in linked list stack
next pointer	Links nodes in linked list	**malloc**	Dynamic memory allocation (C)
free	Release allocated memory	**Delimiter**	Bracket/parenthesis in expressions
Operand	Value/variable in expression	**Operator**	Symbol for operation (+, –, *, /)
Traversal	Visiting every element: O(n)	**Linear DS**	Elements in sequence (1D)
Static	Fixed size, compile-time	**Dynamic**	Variable size, runtime allocation
Undo Stack	Stores actions for undo operation	**Redo Stack**	Stores undone actions for redo
Balance Check	Verify matching brackets	**Reversal**	Stack reverses order of elements

11 Formula / Syntax Reference Box

CORE FORMULAS

```
// Stack empty condition
top == -1

// Stack full condition (array)
top == MAX - 1

// Push logic
stack[++top] = x;
// equivalent to:
top = top + 1;
stack[top] = x;

// Pop logic
val = stack[top--];
// equivalent to:
val = stack[top];
top = top - 1;

// Peek logic
return stack[top];

// Current size of stack
size = top + 1;
```

TRAVERSAL & UTILITY

```
// Traverse all elements (top to bottom)
while (top != -1) {
    print(stack[top]);
    top--;
}

// Traverse without modifying (peek)
for (i = top; i >= 0; i--)
    print(stack[i]);

// Linked list stack traversal
Node* temp = top;
while (temp != NULL) {
    print(temp→data);
    temp = temp→next;
}

// postfix evaluation snippet
b = pop(); a = pop();
push(a op b);
```

12 Problem-Solving Patterns

Reverse a String / Stack

```
reverseString(s):
  stack ← empty
  for each char c in s:
      push(c)
  result ← ""
  while !isEmpty():
      result += pop()
  return result

// "HELLO" → push H,E,L,L,O
// pop O,L,L,E,H → "OLLEH"
```

Next Greater Element

Recursion Simulation via Stack

```
// Factorial using explicit stack
factorial(n):
  stack ← empty
  while n > 1:
      push(n)
      n--
  result ← 1
  while !isEmpty():
      result *= pop()
  return result
```

Sort a Stack

```
nextGreater(arr, n):
  stack ← empty
  result ← [-1, -1, ..., -1]
  for i = n-1 downto 0:
    while !isEmpty() AND
          stack.top() <= arr[i]:
        pop()
    if !isEmpty():
        result[i] = stack.top()
    push(arr[i])
  return result
```

```
// Insert element in sorted order
sortedInsert(stack, x):
  if isEmpty() OR x > peek():
      push(x)
  else:
      temp ← pop()
      sortedInsert(stack, x)
      push(temp)

sortStack(stack):
  if !isEmpty():
      temp ← pop()
      sortStack(stack)
      sortedInsert(stack, temp)
```

🚀 PATTERN RECOGNITION

Use Stack when:

- Need reversal of order
- Need to "remember" past
- Matching/balancing pairs

Key indicators:

- LIFO access needed
- Expression evaluation
- Backtracking required

Common patterns:

- Push then process on pop
- Use two stacks for queue
- Stack + recursion = DFS

13 Flashcards — 35 Q&A Cards

Q: What is a Stack?

A: A linear data structure following the LIFO (Last In, First Out) principle

Q: What does LIFO stand for?

A: Last In, First Out — last element pushed is first to be popped

Q: What is the PUSH operation?

A: Insert an element at the TOP of the stack

Q: What is the POP operation?

A: Remove and return the element at the TOP of the stack

Q: What is PEEK / TOP?

A: View the top element without removing it — O(1)

Q: What is Overflow?

A: Trying to push into a full stack — condition: top == MAX−1

Q: What is Underflow?

A: Trying to pop from an empty stack — condition: top == −1

Q: Initial value of top?

A: top = −1 (indicates empty stack)

Q: Time complexity of Push?

A: O(1) — constant time, direct access to top

Q: Time complexity of Pop?

A: O(1) — constant time, direct access to top

Q: Time complexity of Search?

A: O(n) — must scan all elements in worst case

Q: Space complexity of Stack?

A: O(n) — n elements stored

Q: Advantage of Array Stack?

A: Simple implementation, fast O(1) access, no pointer overhead

Q: Disadvantage of Array Stack?

A: Fixed size — overflow possible, memory may be wasted

Q: Advantage of Linked List Stack?

A: Dynamic size — no overflow, efficient memory usage

Q: Disadvantage of Linked List Stack?

A: Extra pointer overhead, slower due to dynamic allocation

Q: What is Infix expression?

A: Operator placed between operands: A + B

Q: What is Postfix expression?

A: Operator placed after operands: AB+

Q: What is Prefix expression?

A: Operator placed before operands: +AB

Q: Convert A + B * C to Postfix?

A: ABC*+ (multiplication has higher precedence)

Q: Evaluate postfix: 5 3 + 2 *

A: (5+3)*2 = 8*2 = 16

Q: What is a Stack Frame?

A: Memory block created per function call, holding locals and return address

Q: How does recursion use a stack?

A: Each recursive call pushes a frame; returning pops it

Q: isEmpty condition?

A: top == −1 (array) or top == NULL (linked list)

Q: isFull condition?

A: top == MAX – 1 (array only; linked list rarely full)

Q: Stack application in DFS?

A: Push unvisited neighbors; pop to explore next node

Q: How to reverse a string with stack?

A: Push all chars, then pop all — LIFO reverses order

Q: Balanced parentheses algorithm?

A: Push opening brackets; pop and match on closing brackets

Q: Operator precedence order?

A: ^ (highest) > * / > + – (lowest)

Q: What is a Node in LL stack?

A: struct with int data and struct Node* next pointer

Q: Stack used in undo/redo?

A: Two stacks — undo stack and redo stack; push/pop actions

Q: Next Greater Element approach?

A: Traverse right to left, use stack to track pending answers

Q: Pop order of push sequence 1,2,3?

Q: Can stack be implemented with queue?

A: 3, 2, 1 — LIFO reverses the push order

A: Yes — using two queues with O(n) push or O(n) pop

Q: Stack vs Queue main difference?

A: Stack = LIFO (one end); Queue = FIFO (two ends)

14 Exam Tips — Mistakes, Strategies & Writing Guide

❌ COMMON MISTAKES TO AVOID

- Forgetting overflow/underflow checks in push/pop algorithms
- Incorrect top update — incrementing BEFORE assigning (Push) and decrementing AFTER reading (Pop)
- Confusing infix and postfix notation in conversions
- Forgetting operator precedence during Infix→Postfix conversion
- Using `top == 0` for empty check instead of `top == -1`
- Not mentioning time complexity in algorithm questions
- Confusing left/right operand order in postfix evaluation (pop b first, then a)
- Not freeing memory in linked list Pop (memory leak)
- Drawing wrong stack diagram — top should be at the TOP visually
- Forgetting to handle the case when expression ends — flush remaining operators from stack

✅ WRITING TIPS FOR EXAMS

- **Always draw a stack diagram** — visual aids score extra marks
- **Show step-by-step push/pop** trace for every operation question
- **Mention complexity** (O(1) / O(n)) after every algorithm
- **Write overflow/underflow** conditions explicitly in pseudocode
- **Use proper formatting** — indent code, use arrows for diagrams
- **For expression conversion**, trace a table with Symbol, Stack, Output columns
- **Mention applications** whenever asked "what is a stack" — shows depth
- **Write LIFO in full** on first use: Last In, First Out

🎯 HIGH-VALUE TOPICS (STUDY FIRST)

- 🥇 Infix → Postfix conversion with trace table
- 🥇 Push/Pop algorithms with overflow/underflow
- 🥇 Postfix evaluation with example
- 🥈 Balanced parentheses algorithm
- 🥈 Array vs Linked List comparison table
- 🥉 Applications of stacks
- 🥉 Complexity of all operations

How to Answer "Explain Push Operation"

1. Give one-line definition of Push
2. State the overflow condition
3. Write the algorithm (step-by-step)
4. Draw a before/after diagram
5. Write the pseudocode
6. State the time complexity: O(1)

How to Answer Expression Conversion

1. State the expression type (Infix/Postfix)
2. Write the algorithm rules
3. Draw the trace table: Symbol | Stack | Output
4. Show final result clearly
5. Verify by evaluating both expressions

15 Quick Revision Sheet — Last-Minute Reference

DEFINITION

- Stack = Linear + LIFO
- Access only at TOP
- top = -1 when empty
- top = MAX-1 when full

CONDITIONS

- Empty: top == -1
- Full: top == MAX-1
- Overflow → push on full
- Underflow → pop on empty

COMPLEXITIES

- Push → O(1)
- Pop → O(1)
- Peek → O(1)
- isEmpty → O(1)
- Search → O(n)
- Traverse → O(n)
- Space → O(n)

PUSH PSEUDOCODE

```
if top == MAX-1:
    OVERFLOW
else:
    top ← top + 1
    stack[top] ← x
```

POP PSEUDOCODE

```
if top == -1:
    UNDERFLOW
else:
    val ← stack[top]
    top ← top - 1
    return val
```

PEEK PSEUDOCODE

```
if top == -1:
    return "Empty"
return stack[top]
```

APPLICATIONS

- Expression conversion
- Postfix evaluation
- Balanced brackets
- Function call stack
- Undo/Redo
- DFS / Backtracking
- Reversal

PRECEDENCE

- ^ → highest (3)
- * / → medium (2)
- + - → low (1)
- ( ) → lowest (0)

CONVERSIONS

- A+B → AB+ (postfix)
- A+B → +AB (prefix)
- A+B*C → ABC*+
- (A+B)*C → AB+C*

ARRAY VS LINKED LIST (KEY DIFFERENCES)

Feature	Array	Linked List
Size	**Fixed**	**Dynamic**
Overflow	**Possible**	**Rare**
Memory	Contiguous	Scattered
Overhead	**None**	**Pointers**

INFIX→POSTFIX RULES

- Operand → output directly
- (→ push to stack
-) → pop till (, discard (
- Operator → pop higher/equal precedence, then push
- End → pop all remaining

POSTFIX EVALUATION RULES

- Number → push
- Operator → pop b, pop a, push (a op b)
- Final stack top = answer

★ ALL PRIMARY STACK OPERATIONS ARE O(1) ★ ONLY SEARCH & TRAVERSAL ARE O(n) ★

Stack = LIFO · Top = only access point · Overflow = full · Underflow = empty